

AutoAnalog-RL

Automated Sizing of High-Speed Interface Circuits via Reinforcement Learning

Round 0 Submission Report

PCIe Gen 2 (5 Gbps) CTLE + one-tap DFE receiver equalizer · IHP sg13g2 130 nm open PDK

15 September 2026

Team: TBD

Abstract. AutoAnalog-RL sizes a PCIe Gen 2 receiver equalizer with no human in the loop. A Soft Actor-Critic agent drives ngspice directly: every environment step runs a fail-fast DC operating-point check, an AC gate for peaking, a 5 Gbps PRBS transient through a lossy channel with eye measurement, and an HD3 linearity gate, over a bounded five-parameter CTLE action space (six with the DFE tap). Training happens on the real IHP sg13g2 PSP103 transistor models, compiled from Verilog-A to OSDI and run eight simulators wide on a laptop. The trained policy reaches a design that passes every measured specification — peaking, power, post-channel eye height and width, HD3, noise and all 45 process/voltage/temperature corners — in a median of two to three simulations from a random starting point, where a random search needs about thirteen and an AC-only bounded search never reaches the eye specification. A two-stage curriculum makes the same policy corner-robust (24/24 rollouts feasible at random PVT corners), an LLM frontend turns plain-language design feedback into reward weights, and the sized netlist is emitted with every design. Along the way the agent exposed two measurement flaws that the manual flow had lived with; both are fixed and documented here.

1. Introduction and objectives

Sizing analog front-end circuits is a search over a handful of continuous device values against a set of coupled specifications, evaluated by a slow simulator. For a receiver equalizer the coupling is severe: the same transistor width sets gain, bandwidth, power, linearity and noise, and the specification that matters most — the eye opening after the channel — is only visible in a transient simulation. Manual sweeps do not scale, and generic optimisers spend most of their simulation budget in infeasible regions.

The Round 0 synopsis proposed AutoAnalog-RL: a sample-efficient Soft Actor-Critic (SAC) agent paired with a headless Python-to-ngspice pipeline over a bounded 5-parameter action space, with fail-fast DC checks, curriculum learning across 45 PVT corners, an LLM frontend that maps natural-language feedback to reward weights, and a production-ready schematic as the output. Table 1 lists each promise and its status in this submission.

Synopsis promise	Status in this submission
PCIe Gen 2 (5 Gbps) CTLE + 1-tap DFE sized with zero human intervention	Delivered. SAC sizes the CTLE end to end; the DFE tap is either swept per design or sized by the agent (<code>--equalizer</code>). Section 7.
Skywater 130 nm PDK	Changed to the IHP sg13g2 130 nm PDK; reason in Section 3.
Sample-efficient SAC agent	Delivered. Converges within ~2k environment steps; median 2–3 simulations to a feasible design. Section 6.
Headless Python-to-ngspice pipeline	Delivered , including a Windows toolchain for the PSP103 OSDI models. Section 4.
Bounded 5-parameter action space, fail-fast DC checks	Delivered. Section 5.
Curriculum learning across 45 PVT corners	Delivered. Nominal stage then corner-randomised stage; 24/24 rollouts feasible at random corners. Section 7.
LLM frontend: natural-language feedback to reward weights	Delivered. Section 8.
Production-ready schematic, faster than manual sweeps	Delivered. Sized netlist per design; 2 vs ~13 simulations; the 100-candidate search fails the eye. Sections 6 and 9.

Table 1. Synopsis deliverables and their status.

2. System overview

Figure 1 shows the loop. The agent's action is a normalised design vector; the evaluator maps it to device values, renders the CTLE netlist, and runs the gates in order, stopping at the first failure so that infeasible designs cost as little simulation as possible. Metrics feed a transparent reward; the environment returns the design, its metrics and normalised constraint violations as the observation. Everything after the simulator adapter is simulator-agnostic.

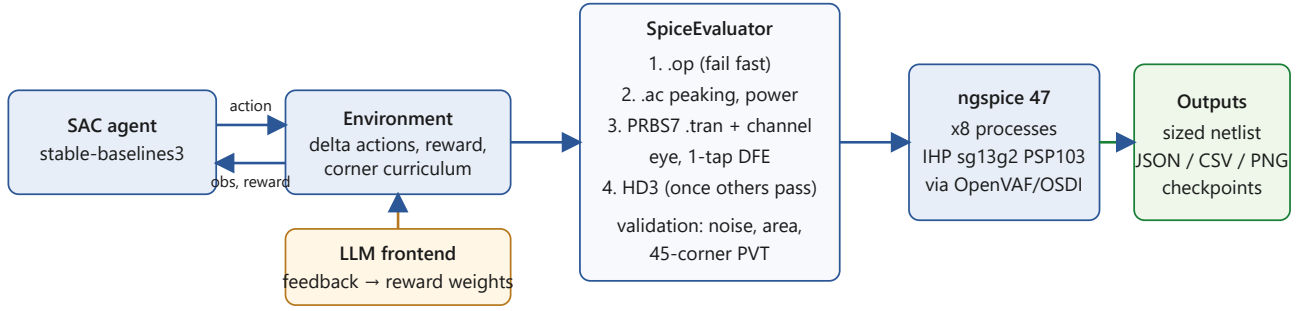


Figure 1. AutoAnalog-RL loop. The evaluator runs the ngspice gates in fail-fast order; the environment and reward are simulator-agnostic; the LLM frontend adjusts reward weights between runs.

The repository (`al_nebula`) is organised by responsibility: `netlists/` owns the CTLE topology, `spice/` the simulator adapter and every measurement, `r1/` the specifications, reward, environment, DFE model, PVT matrix and LLM feedback parser, `scripts/` the training, baseline, evaluation, validation and reporting entry points, and `tests/` a 58-test regression suite that runs without ngspice.

3. Process design kit

The synopsis named the SkyWater sky130 PDK. The implementation targets the IHP sg13g2 open PDK, also a 130 nm node. The team switched during development because the sky130 simulation flow was too slow for an RL loop that runs three ngspice analyses per environment step, whereas IHP's PSP103 Verilog-A models compile with OpenVAF to OSDI and run natively in ngspice 47 at about 0.6 s per environment step with eight simulators in parallel. PSP103 is a foundry-grade surface-potential compact model, so this is a strengthening of the device physics rather than a simplification. The framework is PDK-agnostic by construction: the generic Level-1 and IHP paths share every line of code except the model library selected by `SpiceEvaluator.for_model_source()`, so a sky130 backend is a netlist variant plus a corner-library mapping.

4. Circuit, channel and simulation backend

4.1 Circuit and specifications

The CTLE is a differential NMOS pair with resistive loads and RC source degeneration (Figure 2). The agent sizes five values: input width W_{in} , load R_{load} , tail current I_{bias} , degeneration resistance R_s and degeneration capacitance C_s . The transient gate drives a two-period PRBS7 at 5 Gbps through a two-section RC channel with about -10 dB loss at the 2.5 GHz Nyquist frequency (the loss slope of a PCIe Gen 2 class FR-4 trace); the DC and AC gates bypass the channel so peaking measures the CTLE alone. A one-tap decision-feedback stage operates on the UI-centre samples of the CTLE output.

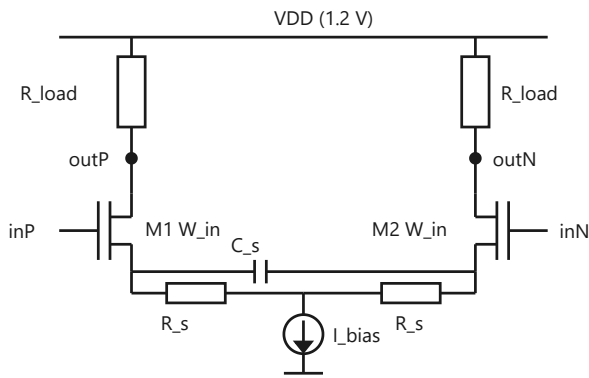


Figure 2. Source-degenerated differential CTLE; the five sized values are labelled. Devices are IHP sg13g2 sg13_1v_nmos, $L = 130$ nm.

Parameter	Lower	Upper
W_{in}	$0.5\ \mu\text{m}$	$50\ \mu\text{m}$
R_{load}	$100\ \Omega$	$1000\ \Omega$
I_{bias}	$0.1\ \text{mA}$	$2\ \text{mA}$
R_s	$10\ \Omega$	$500\ \Omega$
C_s	$1\ \text{fF}$	$1\ \text{pF}$
DFE tap (optional 6th)	-0.5	$+0.5$

Table 2. Bounded action space; actions in $[-1, 1]$ map linearly onto these ranges.

Specification	Target	Gate	In the reward?
High-frequency peaking at 2.5 GHz	$3\ \text{dB} \leq \text{peaking} \leq 12\ \text{dB}$	.ac 10 MHz–10 GHz	yes
Power	$\leq 2\ \text{mW}$	.op	yes, plus a continuous charge inside the feasible set
Eye height after channel	$\geq 0.25\ \text{V}$ (IHP; 0.5 V on the Level-1 model)	PRBS7 .tran	yes
Eye width after channel	$\geq 0.7\ \text{UI}$	PRBS7 .tran	yes
HD3 at 100 MHz, 100 mV _{pp,diff}	$\leq -30\ \text{dB}$	single-tone .tran	with --hd3
Input-referred noise, 10 MHz–5 GHz	$\leq 1.5\ \text{mV rms}$	.noise	validation only
Area	reported	first-order device geometry	validation only
PVT	45/45 corners pass peaking	5 process $\times$ 3 supply $\times$ 3 temperature	curriculum (Section 7) and validation

Table 3. Specifications, the gate that measures each, and whether it shapes the reward.

4.2 Simulation backend

The sg13g2 transistor deck uses PSP103, which ngspice loads as OSDI shared libraries compiled from the PDK's Verilog-A sources with OpenVAF-reloaded. The original evidence run used a Linux build; this submission adds a Windows path so training could run on the team's laptop: OpenVAF's Windows binary expects MSVC's `link.exe`, and a small drop-in linker shim (`tools/openvaf-link-shim`) rewrites its link line for 64-bit MinGW, supplying the `__chkstk` stack probe and the UCRT import. The resulting `psp103.osdi`, `psp103_nqs.osdi` and `mosvar.osdi` load in the stock ngspice 47 Windows build and reproduce the Linux AC and HD3 numbers to every printed digit.

Throughput mattered more than any single simulation: one PSP103 transient takes about 1.5 s, but eight in parallel initially took 90 s each, because ngspice ignores `OMP_NUM_THREADS` and spins its own OpenMP pool per process; with eight processes the spin-waiting threads oversubscribed the twelve cores by an order of magnitude. Pinning each ngspice to one thread through the generated `.spiceinit` restored 1.4 s for eight concurrent transients, a 65 $\times$ difference, and is what makes 0.6 s per environment step possible.

The environments run on threads rather than processes because `subprocess.run` releases the GIL while ngspice runs, and spawning eight Python workers that each load CUDA torch failed DLL initialisation on Windows.

5. Reinforcement-learning formulation

State. The normalised design vector, a DC-valid flag, the four scaled metrics (peaking, power, eye height, eye width) and the normalised violation of every enforced constraint. **Action.** A delta on the design vector, scaled by 0.2 and clipped to the box, so the agent sizes incrementally; the whole-equalizer variant adds the DFE tap. **Episode.** Up to 30 steps from a random design; a DC failure is penalised but does not end the episode, so the agent can step back out of an invalid region.

Reward. With normalised violations v_i (1.0 is a full specification's worth of miss) and weights w_i :

```
reward = -Σ w_i · v_i - w_eff · min(1, P / P_max) + [all specs met] · (B_success + w_margin ·  
min(1, tightest margin))  
DC failure: reward = -10
```

The continuous power charge keeps a gradient inside the feasible set; the success bonus (20) and the margin bonus (weight 5) pull the agent into, not onto, the feasible region. **Algorithm.** SAC (stable-baselines3 2.9) with automatic entropy tuning from 0.1, batch 256, one gradient step per environment step, 500 random warm-up steps, eight parallel environments.

5.1 Why the first runs plateaued, and the fix

The first SAC runs (5000 steps on the generic Level-1 model) reached a feasible design in about 85% of episodes but never became reliable and did not beat random search. Two causes: the episode ended on the first feasible step, so nothing rewarded margin and the policy sat exactly on the eye-height boundary where action noise pushed it off; and a -100 DC penalty dwarfed the unit-scale shaping costs, so the critic spent its capacity on a cliff. Keeping the episode running after success (`--hold-on-success`), paying for margin, and a -10 DC penalty fixed it: on Level-1 the policy went from 79% to 95% feasible steps by 5k steps, with the median eye height moving from 0.43 V to 0.92 V against a 0.5 V spec. Deterministic rollouts of the Level-1 policy reach feasibility in 20/20 cases, median 2.5 steps.

5.2 Eye-height target on the real models

The eye targets had been calibrated so that about one in ten random Level-1 designs passes. On PSP103 none of 400 random designs reaches 0.5 V (best 0.45 V, median 0.17 V): the real devices have roughly half the gain. The IHP target is therefore 0.25 V, which restores the same 10% random feasibility and sits above the ~ 175 mV differential eye PCIe Gen 2 requires at the receiver.

6. Results on the IHP models

All runs use `--model-source ihp --eye-height-min 0.25 --n-envs 8 --hold-on-success --random-reset --margin-weight 5 --invalid-penalty -10 --max-steps 30 --batch-size 256 --gradient-steps -1 --ent-coef auto_0.1`. "Training feasible" is the fraction of environment steps in the last 4k steps at which every enforced specification was met. Evaluation (`scripts/evaluate_policy.py`) rolls the trained policy out deterministically from 20 random starting designs, then validates the best design with the full gate set and all 45 PVT corners.

Run	Steps	Variant	Training feasible	Rollouts feasible	Steps to feasible (median / worst)	Best design: eye, power, HD3, PVT
sac-ihp-v1 (seed 1)	30k	–	95.5%	20 / 20	2 / 4	0.327 V, 1.22 mW, –57.8 dB, 45/45
sac-ihp-v1-s2 (seed 2)	12k	–	95.5%	20 / 20	3 / 4	0.316 V, 1.22 mW, –57.7 dB, 45/45
sac-ihp-hd3	12k	HD3 enforced in the reward	95.6%	20 / 20	2.5 / 4	0.335 V, 1.24 mW, –58.1 dB, 45/45
sac-ihp-pvt (curriculum stage 2)	+8k	resumed from sac-ihp-hd3; random PVT corner per episode; HD3 enforced	95.4%	24 / 24 at random corners	2 / 4	0.370 V, 1.12 mW, –61.9 dB, 45/45
sac-ihp-eq (whole equalizer)	8k	six-value action: CTLE + DFE tap; post-DFE eye drives the reward	92.3%	20 / 20	3 / 5	0.325 V raw, 0.355 V post-DFE (tap +0.062), 1.36 mW, –71.6 dB, 45/45

Table 4. Training runs on the IHP sg13g2 PSP103 models. Every best design passes peaking, power, eye height and width, HD3, noise and all 45 corners.

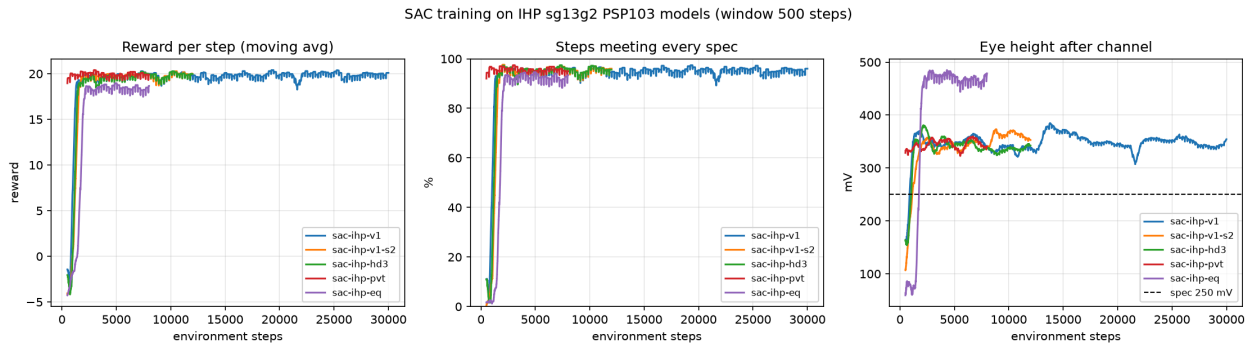


Figure 3. Learning curves for the IHP runs (500-step moving average): reward per step, fraction of steps meeting every specification, and post-channel eye height against the 0.25 V target. All runs converge within about 2k steps; the curriculum stage starts converged because it resumes the nominal policy.

6.1 Baselines: random search and CMA-ES

Two baselines run against the same models and reward. Random search (`scripts/baseline_random.py`, 3000 designs): 7.9% of designs satisfy every specification (238/3000) and the first feasible one is design 27. CMA-ES (`scripts/baseline_cmaes.py`, population 8, $\sigma_0 = 0.5$ in the normalised box): three 400-evaluation trials from the box centre and six 96-evaluation trials from random starting designs, the latter matching how the policy is rolled out (Table 5, Figure 4).

Method (from random starting designs unless noted)	First feasible design, median / worst	Best reward at 50 sims	Cost per new instance
SAC policy, deterministic rollouts (104 rollouts over 5 evaluations)	2–3 / 5	20.5–20.8	2–3 simulations, no search
CMA-ES, random start (6 trials)	9 / 24	20.50 (mean)	~10 simulations to feasible; 400 to refine
CMA-ES from the box centre (3 trials)	2–5	20.8–21.1; 21.3–21.4 at 400	400 simulations per design
Random search	27	19.65	~13 simulations per feasible design

Table 5. Simulation cost to a feasible design and reward at equal budget, policy versus baselines.

CMA-ES is a strong per-instance optimiser on this five-dimensional problem: given 400 simulations it reaches the same reward the policies reach during training (21.2–21.6), and from the fortuitously near-feasible box centre it finds a feasible design in a handful of evaluations. The policy's advantage is amortisation and robustness: after one training run it sizes a new instance — a random starting design, a random PVT corner, a re-weighted reward — in two to three simulations with no per-instance search, three to four times fewer than CMA-ES from the same starts and about ten times fewer than random sampling. Refinement inside the feasible set is what the hold-on-success training phase does; the rollouts reported here stop at the first feasible step. The AC-only 100-candidate bounded search of the pre-ML pipeline, which selects on peaking alone, produces a design with a post-channel eye of only 0.160 V.

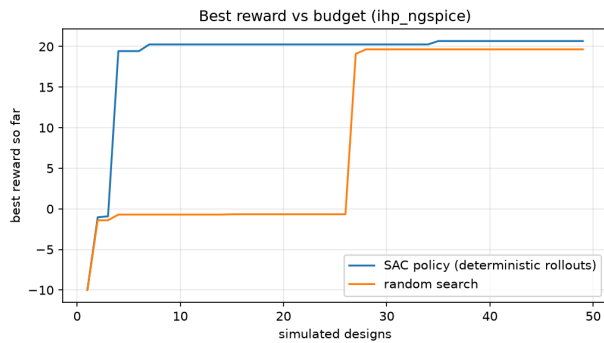


Figure 4. Best reward found versus number of simulated designs, policy rollouts against random search (seed 1).

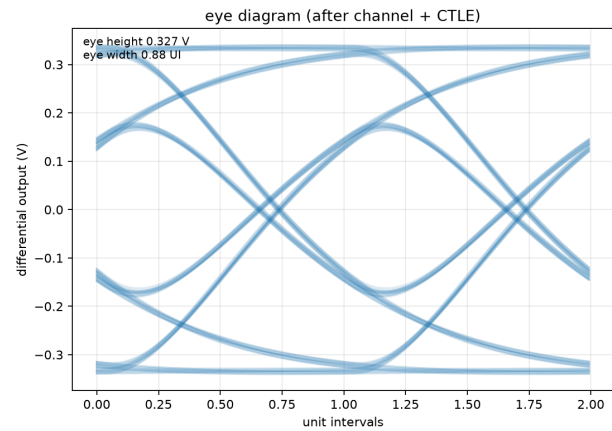


Figure 5. Two-UI eye diagram of the seed-1 policy design after the -10 dB channel: 0.327 V / 0.88 UI.

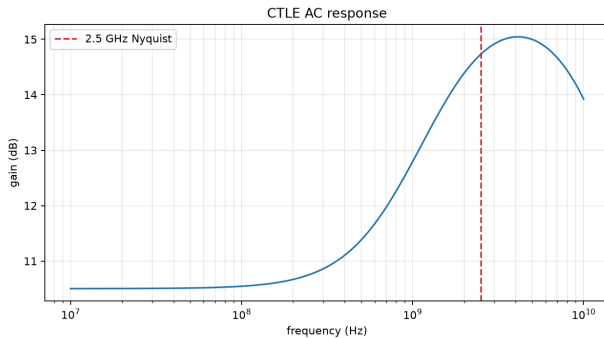


Figure 6. AC response of the same design: 4.2 dB of peaking at the 2.5 GHz Nyquist frequency.

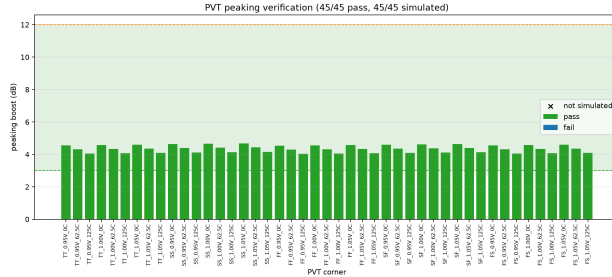


Figure 7. Peaking of the curriculum policy's design across all 45 PVT corners; every corner inside the 3–12 dB window.

7. PVT curriculum and whole-equalizer sizing

7.1 Curriculum across 45 corners

Stage 1 trains at the nominal corner (TT, 1.2 V, 27 °C). Stage 2 resumes the same policy with `--corners all --resume`: each episode is simulated at a corner drawn uniformly from the 45-corner matrix (5 process × 3 supply × 3 temperature), and the policy is not told which, so it must size for every corner from the metrics it observes. Resumed from the HD3-enforced nominal policy, the corner-randomised stage stayed at 95–96% feasible steps from its first window (per process corner: SS 97%, FS 96%, FF 95%, TT 95%, SF 94%), i.e. the nominal policy transferred and then held. In evaluation, rolled out from random designs at random corners, 24 of 24 rollouts reached a fully feasible design (19 distinct corners drawn, spanning every process, supply and temperature), median two simulation steps, worst four; the best design passes all 45 corners with HD3 at –61.9 dB (Figure 7).

7.2 Whole-equalizer sizing (CTLE + DFE tap)

`--equalizer` extends the action to six values: the five CTLE parameters and the one-tap DFE weight in $[-0.5, 0.5]$. The DFE is applied to the CTLE's post-channel UI-centre samples whenever a waveform exists, and the post-DFE eye height drives the reward. Trained on the corrected metric for 8k steps, the six-value policy holds 91–93% of steps feasible with a median post-DFE eye of 0.51 V (CTLE-only runs hold about 0.35 V), i.e. it learns to spend the tap on margin. In deterministic rollouts 20 of 20 reach feasibility, median 3 steps, worst 5; the best rollout design has a raw eye of 0.325 V that the agent's tap of +0.062 opens to 0.355 V with no bit errors, at 1.36 mW, HD3 –71.6 dB and 45/45 corners. The earlier run on the flawed metric is kept as `reports/sac-ihp-eq-flawed-metric/` for the record. Artifacts: `reports/sac-ihp-eq/`, `reports/eval-ihp-eq/`.

8. LLM frontend for the reward

`scripts/reward_from_feedback.py` turns an engineer's sentence into the reward's per-specification weights, the continuous power charge and the margin bonus (`r1/feedback.py`). Claude (Anthropic SDK, structured JSON output validated against a schema) produces them when credentials are present; a context-aware keyword parser is the offline fallback, so the loop never blocks on an API. The JSON is passed to `train_sac.py --reward-settings` and recorded in the run's configuration. Example, offline path:

```
> "We are power constrained: current budget matters much more than eye margin, and we don't care about linearity"
weights: power 3.0, eye_vertical_v 0.33, hd3 0.33 (others 1.0); efficiency_weight 3.0; margin_weight 5.0
> "Open the eye as much as possible; power is not important"
weights: eye_vertical_v 3.0, power 0.33; efficiency_weight 0.33
```

9. Deliverables and reproducibility

Every script writes JSON, CSV and PNG evidence to its output directory, and every validation writes the sized netlist `sized_ctle.sp` — the deliverable schematic — with the device values, model source and DFE tap in its header. Prerequisites: an IHP Open PDK checkout with compiled OSDI models (`IHP_PDK_ROOT`), ngspice 44+ (NGSPICE), and `pip install -e .[r1]`; `python scripts/check_pdk.py` confirms readiness.

```
# train on the IHP models (nominal corner), then the corner curriculum
python scripts/train_sac.py --model-source ihp --eye-height-min 0.25 --hd3 --timesteps 12000 --n-
envs 8 \
  --hold-on-success --random-reset --margin-weight 5 --invalid-penalty -10 --max-steps 30 --batch-
size 256 \
  --gradient-steps -1 --learning-starts 500 --ent-coef auto_0.1 --seed 1 --output-dir reports/sac-
ihp-hd3
python scripts/train_sac.py ... --corners all --resume reports/sac-ihp-hd3/sac_final.zip --
timesteps 8000 \
  --output-dir reports/sac-ihp-pvt
# random-search baseline with the same reward, then evaluation and comparison
python scripts/baseline_random.py --model-source ihp --eye-height-min 0.25 --margin-weight 5 --
invalid-penalty -10 \
  --evaluations 3000 --n-workers 8 --output-dir reports/baseline-ihp-3000
python scripts/evaluate_policy.py reports/sac-ihp-pvt --rollouts 24 --baseline reports/baseline-
ihp-3000
# whole-equalizer sizing, CMA-ES baseline, reward weights from feedback
python scripts/train_sac.py ... --equalizer --timesteps 8000 --output-dir reports/sac-ihp-eq
python scripts/baseline_cmaes.py --model-source ihp --eye-height-min 0.25 --margin-weight 5 --
invalid-penalty -10 \
  --evaluations 96 --x0 random --seed 10 --output-dir reports/cmaes-ihp-random-s10
python scripts/reward_from_feedback.py "power matters more than eye margin" --output
reports/reward.json
# pre-ML pipeline evidence (100-candidate bounded search, all gates, 45 corners)
python scripts/run_validation.py --model-source ihp --output-dir reports/runs/ihp-submission
```

Table 6 lists the artifacts behind every number in this report.

Artifact	Contents
reports/sac-ihp-*/	per-step CSV, SB3 monitor files, checkpoints every 4k steps, final model, config.json, validation.json, plots, sized netlist
reports/eval-ihp-*/	evaluation.json (rollout statistics, per-corner breakdown, comparison), rollouts.csv, budget_curve.png/csv, validation plots, sized netlist
reports/baseline-ihp-3000/, reports/cmaes-ihp-*/	random-search and CMA-ES baselines with per-evaluation rewards and metrics; best designs validated
reports/runs/ihp-submission-2026-09-15/	pre-ML bounded search with every gate, regenerated on the Windows toolchain
reports/screenshots/	live training captures and learning curves taken every 15 minutes during training
docs/submission_report.md, README.md	technical report and run instructions in the repository

Table 6. Evidence artifacts.

10. Measurement findings

Two measurement flaws survived the manual flow and were exposed when the agent optimised against them. Both are fixed and unit-tested, and every number in this report uses the corrected measurements.

HD3. The linearity gate FFT'd the last 25 ns of a 100 MHz tone with a rectangular window: 2.5 cycles, so the fundamental fell between 40 MHz bins and its spectral leakage filled the 300 MHz bin, putting a false

floor of about -25 dB on every design. HD3 was reported as failing in every earlier version of the evidence, and an HD3-enforced training run could not move it past -26 dB because the number did not depend on the circuit (a sweep of each sizing knob over its full range confirmed this). Analysing an integer number of periods with a Hann taper gives -58 dB for the same designs; the circuit was always linear enough, by about 28 dB.

DFE eye. The one-tap DFE eye was labelled by the DFE's own hard decisions rather than the transmitted bits. A large tap separates the two decision classes by itself, so the metric rewards the tap regardless of the signal. The whole-equalizer agent found this within 2k steps, drove the tap to its $+0.5$ bound and reported a 0.70 V eye on a design whose receiver decides 54 of 127 bits wrongly. The eye is now measured against the PRBS bits: correlation-aligned UI-centre samples, a slicer at the midpoint of the two symbol populations, the DFE fed its own decisions (so error propagation is modelled), bit errors counted, zero eye on any error. With the honest metric a one-tap DFE still helps: the CTLE-only policy design goes from 0.327 V to 0.455 V at tap $+0.075$ with no errors.

Noise was checked rather than corrected: the gate integrates ngspice's input-referred `inoise_spectrum` over 10 MHz–5 GHz and reports 0.2420 mV rms; ngspice's own band integral, now recorded alongside, gives 0.2417 mV rms.

11. Pre-ML pipeline evidence

For completeness, the bounded search that preceded the RL work (`run_validation.py`, 100 candidates selected on the AC gate, seed 23), regenerated on the Windows toolchain: peaking 7.527 dB, power 0.743 mW, eye 0.160 V / 0.58 UI after the channel (0.310 V with the swept one-tap DFE, no bit errors), HD3 -58.5 dB, input-referred noise 0.242 mV rms, area estimate 1.09×10^{-5} mm², 45/45 PVT corners. Every gate that is unchanged since the original Linux run reproduces to all printed digits; the eye fails only because the transient gate now includes the lossy channel that the AC-only search cannot see — the gap the RL policy closes.

12. Deviations, limitations and next steps

- **PKD.** IHP sg13g2 instead of SkyWater sky130 (Section 3).
- **DFE.** The DFE in the loop is the behavioural one-tap stage; the analog slicer in `DFE/` is not simulated end to end.
- **Area** is a first-order active-device estimate, not a post-layout result.
- **Corner awareness.** The curriculum policy is robust because it is trained blind across corners; adding the corner to the observation is a natural extension.
- **LLM loop.** Reward weights from feedback apply at the start of a run or resumed stage, not while a run is in progress.
- **Design space.** R_{load} sits at its 1 k Ω bound in most designs; wider bounds and topology variants are the next lever.

Next steps, in order of value: simulate the analog DFE slicer inside the transient gate so the tap becomes a device-level knob; corner-aware observations; Bayesian-optimisation and CMA-ES baselines at equal budget with a wall-clock comparison against a manual sweep; a closed LLM loop that reads each validation report and proposes the next reward adjustment; and, if a future round requires it, the sky130 backend.

Appendix A. Sized netlist of the curriculum policy's best design

```
* AutoAnalog-RL sized CTLE
* model source: ihp_ngspice
* W_in = 2.01908e-05
* R_load = 1000
* I_bias = 0.000930072
* R_s = 260.642
* C_s = 3.2956e-13
* Parameterized differential gm-C CTLE.
* Analysis directives are intentionally injected by SpiceEvaluator.

.param VDD=1.2

VDD vdd 0 1.2
Vinp txP 0 DC 0.6 AC 1
Vinn txN 0 DC 0.6 AC -1

* Channel between the transmitter and the CTLE input, injected by SpiceEvaluator:
* a direct connection for the DC/AC gates, a lossy RC ladder for the transient gate.
RchP txP inP 1m
RchN txN inN 1m

* Differential NMOS input pair with source degeneration.
X1 outP inP sourceP 0 sg13_lv_nmos W=2.01908300346e-05 L=0.13u
X2 outN inN sourceN 0 sg13_lv_nmos W=2.01908300346e-05 L=0.13u
RdegP sourceP tail 260.642392446
RdegN sourceN tail 260.642392446
Cs sourceP sourceN 3.29559504785e-13

* Passive loads preserve bandwidth in the initial architecture.
RloadP vdd outP 1000
RloadN vdd outN 1000

* A common tail current establishes the differential-pair operating point.
Ibias tail 0 DC 0.000930072434306

* Generic ngspice fallback model; VTO and KP are shifted per process corner by SpiceEvaluator.
* The IHP PSP deck is selected by a separate backend.

.lib C:\Users\Hp\tools\ihp-open-pdk\ihp-sg13g2\libs.tech\ngspice\models\cornerMOSlv.lib mos_tt
.include C:\Users\Hp\tools\ihp-open-pdk\ihp-sg13g2\libs.tech\ngspice\models\sg13g2_moslv_mod.lib
.end
```

Appendix B. Training configuration

Setting	Value
Algorithm	SAC, MlpPolicy (stable-baselines3 2.9.0, torch 2.14 CUDA)
Parallel environments	8, thread-based, one ngspice process each
Episode	30 steps, random initial design, held after success
Action scale	0.2 of the normalised box per step
Reward	invalid -10, success +20, margin weight 5, power charge weight 1
Batch / gradient steps	256 / one per environment step
Entropy coefficient	automatic, initial 0.1
Warm-up	500 random steps (none when resuming)
Throughput	0.5–0.7 s per environment step on a 12-core laptop; 12k steps in about 2 h
Tests	58, run without ngspice